Another essential feature of the system is the possibility of purchasing books. In this chapter, you will start working on the Order Service application. This new component will interact not only with a database but also with Catalog Service. When you have applications extensively relying on I/O operations such as database calls or interactions with other services like HTTP request/response communications, the thread-per-request model used in Catalog Service begins to expose its technical limits.

In the thread-per-request model, each request is bound to a thread exclusively allocated to its processing. If database or service calls are part of the processing, the thread will send out a request and then block, waiting for a response. During idle time, the resources allocated for that thread are wasted, since they cannot be used for anything else. The reactive programming paradigm solves this problem and improves scalability, resilience, and cost-effectiveness for all I/O-bound applications.

Reactive applications operate asynchronously and in a non-blocking way, meaning that computational resources are used more effectively. That’s a huge advantage in the cloud, since you pay for what you use. When a thread sends a call to a backing service, it will not wait idle, but it will move on to executing other operations. This eliminates the linear dependency between the number of threads and the number of concurrent requests, leading to more scalable applications. With the same amount of computational resources, reactive applications can serve more users than their non-reactive counterparts.

Cloud native applications are highly distributed systems deployed in a dynamic environment where change is a constant and failures can and will happen. What if the service is not available? What happens if the request gets lost on its way to the target service? What if the response gets lost on its way back to the caller? Can we guarantee high availability in this context?